Introduction to Machine Learning Methods

Making Complex Decisions

Haris Gačanin
RWTH Aachen University

Outline

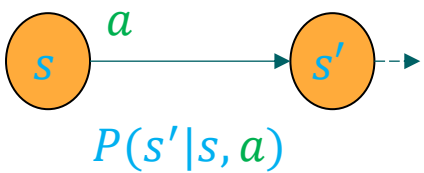
- Markov decision process
- Optimal policy
- Value and Policy iteration

Reference book: Chapter 17.1-17.4

Definition of task environment

Notations – Markov Decision Process (MDP)

- MDP specifies a sequential decision problem for a fully observable, stochastic environment with a Markovian transition model and additive rewards, consisting of
  - A set of states S
  - A set of possible actions A
  - $P(s'|s,a)$ – a transition model
  - $R(s)$ – a reward function



Notations – Utilities

- A utility function $U(s)$ is the mechanism that tells the agent **what is desirable** and **how to evaluate** **different outcomes**
  - Because the problem is sequential, the utility function depends on environment history, e.g., a sequence of states

Notations – Utilities

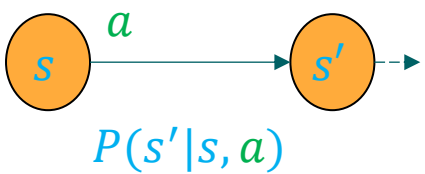
- A utility function $U(s)$ is the mechanism that tells the agent **what is desirable** and **how to evaluate** **different outcomes**
  - Because the problem is sequential, the utility function depends on environment history, e.g., a sequence of states
- A utility function $U(s)$ assigns a value to a state s, where larger values correspond to more preferred outcomes.
  - Goal of the learning agent is to choose actions that maximize the **expected utility** $\mathbf{E}[U(s)]$ as

$$a^* = \arg\max_a \mathbf{E}[U(s')|a]$$

where s' is the future state resulting from action a.

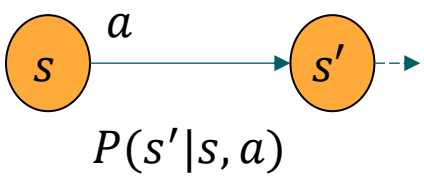
Notations – Transition model

- Transition model, or model, describes the outcome of each action in each state
  - The outcome is stochastic so we denote the probability $P(s'|s,a)$ of reaching state s' from state s after doing action a
  - We assume that the transition model is Markovian process since the the probability of reaching s' from s depends only on s and not on the history of earlier states



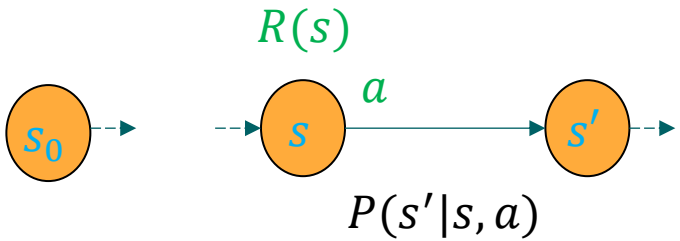
Notation – Environment history

- Environment history is defined by a sequence of states
- For sequential decision making, the utility function depends on a sequence of states rather than on a single state



Notations – Rewards

- Rewards are assigned to states:
  - $R(s)$ denotes the reward an agent receives in state s, which may be positive or negative



Notations – An Action Policy (Solution to an MDP)

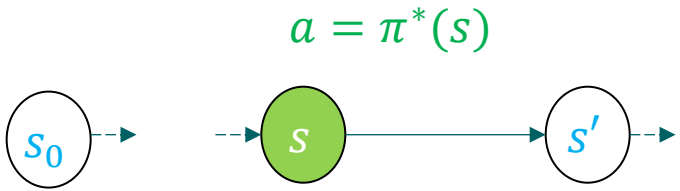
- Since outcomes of actions in reality are **NOT** deterministic, a fixed set of actions cannot be a solution.
- **The policy** $\pi(s)$ specifies an action to be taken in any of the states $s \in S$
  - The policy specifies what the agent should do for any state that it might reach

- If the agent has a complete policy for its environment, then no matter the outcome of any action, the agent will always know what to do next!

- **Note**: Each time a policy is executed from the initial state, the stochastic nature of the environment may lead to a different environment history

Notation – Optimal policy

- The quality of a policy is measured by the *expected* utility of the **possible environment histories** generated by that policy
- An **optimal policy $\pi^*(s)$** is a policy that yields the highest expected utility!
- Given a $\pi^*(s)$, after identifying the state s in which the agent is, the agent search in $\pi^*(s)$ for the next action to take.



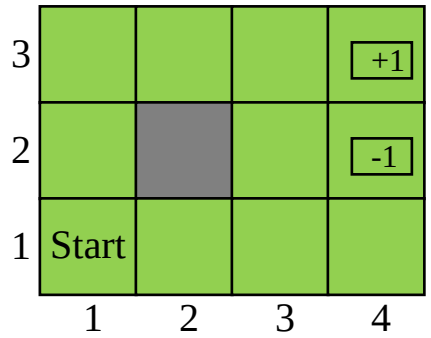
- **Note:** The balance of risk and reward depends on the value of $R(s)$

Example

A sequential decision-making problem

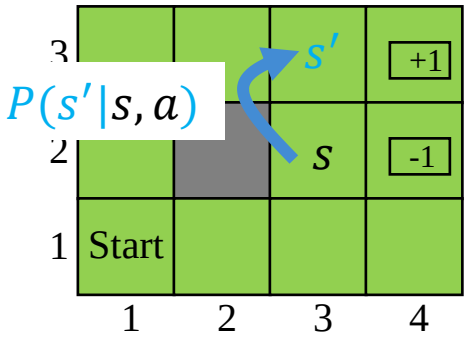
Definitions

- Starting from start state (1, 1), we take an action at each time step from policy $\pi(s)$
  - Possible actions are $A \in \{Up, Down, Left, Right\}$
  - We assume that an action selection mechanism is available!
- Problem terminates when either goal state +1 or -1 is reached.



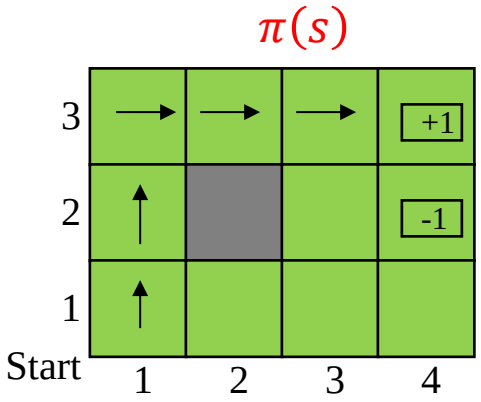
Definitions

- Assume that the environment is fully observable, i.e., the agent always knows where it is.
  - Percepts supply both the current state and the reward received in that state
- Transitions $P(s'|s,a)$: deterministic or stochastic



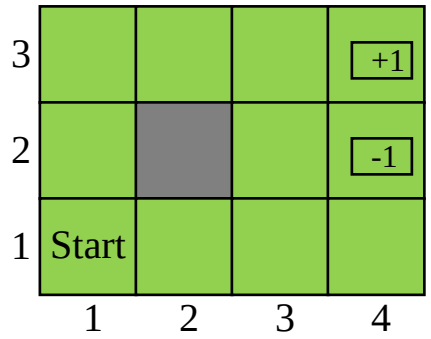
Deterministic environment example

- If the environment is deterministic and the objective is to get the maximum reward the solution is simple:
  - $\pi(s) \in \{Up, Up, Right, Right, Right\}$



Rewards and utilities for deterministic example

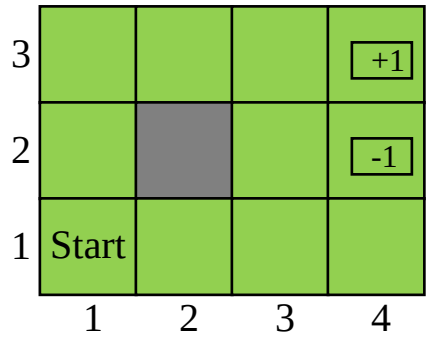
- Reward for all of the states $R(s) = -0.04$ except:
  - Terminal state $R(s = \{4,3\}) = +1$ (Desired Goal state)
  - Terminal state $R(s = \{4,2\}) = -1$ (Not desired state)



Rewards and utilities for deterministic example

- Reward for all of the states $R(s) = -0.04$ except:
  - Terminal state $R(s = \{4,3\}) = +1$ (Desired Goal state)
  - Terminal state $R(s = \{4,2\}) = -1$ (Not desired state)
- The utility function is assumed as the sum of all the visited states:
  - For example, if the agent reaches state (4,3) in 5 steps, the total utility is:

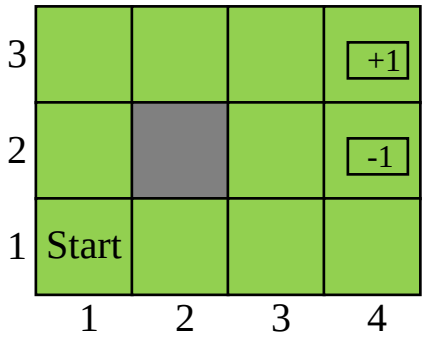
$$U(s = \{4,3\}) = 1 + (5 \times -0.04) = 0.8$$

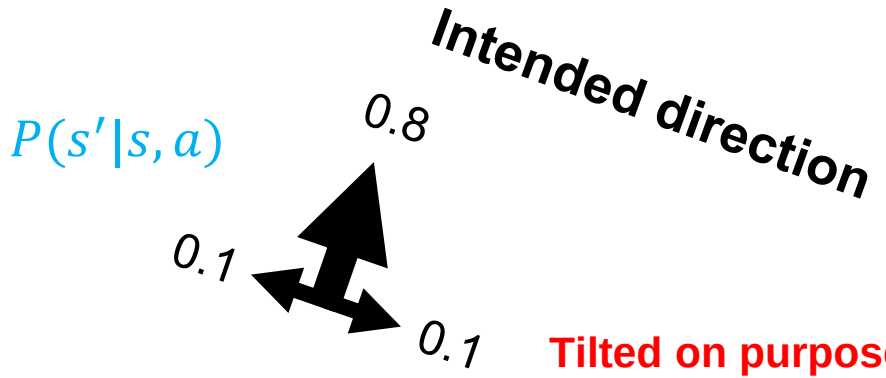


Unfortunately, the environment is not always deterministic!

Stochastic environment example

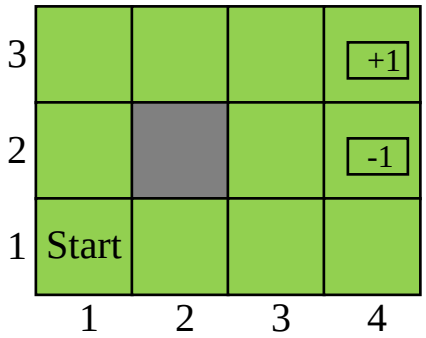
- We introduce stochastic transitions $P(s'|s,a)$ is defined as follows:
  - Transition achieves the intended effect with probability 0.8
  - For 0.2 of the time, the action moves the agent at right angles to the intended direction
  - If the agent hits a wall, it stays in the same square

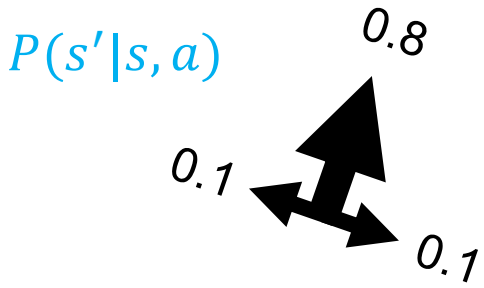




Solution for stochastic environment example

- We focus on policy $\pi(s) \in \{Up, Up, Right, Right, Right\}$ and start from (1,1):
  - The action "*Up*" moves the agent to (1,2) with probability 0.8
  - but with probability 0.1, it moves either right to (2,1) or left, hits into the wall, and stays in (1,1)
    - Thus, $\pi(s)$ reaches the goal state at (4,3) with probability 0.8^5=0.32768

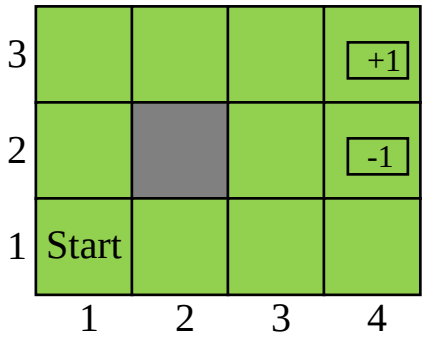




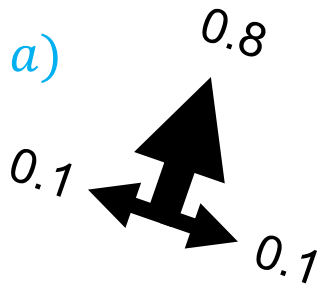
Solution for stochastic environment example

- We focus on policy $\pi(s) \in \{Up, Up, Right, Right, Right\}$ and start from (1,1):
  - The action "*Up*" moves the agent to (1,2) with probability 0.8
  - but with probab… moves either right to (2,1) or left, hits into the wall, and stays in (1,1)
    - Thus, $\pi(s)$ re… …5=0.32768

Now, let's see which squares can be reached from (1,1) by the action sequence $\pi(s)$ and with what probabilities.



$P(s'|s,a)$



Transitions for a stochastic environment example

- If you are in (1,1) there is 80% chance that you will end up in (1,2) if you take action “Up”.

		Intended Action				
		Up	Up	Right	Right	Right
(1,1)	1					
(1,2)		0.8				
(1,3)						
(2,1)						
(2,3)						
…						
(4,3)						

$$P(s = \{4,3\}) = P(a_1) \times P(a_2) \times \cdots = ?$$

Transitions for a stochastic environment example

- If you are in (1,1) there is 80% chance that you will end up in (1,2) if you take action “Up”.
- But, there is 10% chance that you will stay in either (1,1) or move to (2,1) even if you took action Up

		Intended Action				
		Up	Up	Right	Right	Right
(1,1)	1	0.1				
(1,2)		0.8				
(1,3)						
(2,1)		0.1				
(2,3)						
…						
(4,3)						

$$P(s = \{4,3\}) = P(a_1) \times P(a_2) \times \cdots = ?$$

Transitions for a stochastic environment example

- If you are in (1,1) there is 80% chance that you will end up in (1,2) if you take action “Up”.
- But, there is 10% chance that you will stay in either (1,1) or move to (2,1) even if you took action Up
- By following this logic, we obtain the full transition matrix

		Intended Action				
		Up	Up	Right	Right	Right
(1,1)	1	0.1	0.02	0.026	.0284	0.02462
(1,2)		0.8	0.24	0.258	0.2178	0.180542
(1,3)			0.64	0.088	0.0346	0.02524
(2,1)		0.1	0.09	0.034	0.276	0.02824
(2,3)				0.512	0.1728	0.06224
(3,1)			0,01	0,073	0,346	0,02627
(3,2)				0,001	0,0073	0,0443
(3,3)					0,4097	0,17994
(4,1)				0,008	0,0656	0,08672
(4,2)					0,0016	0,014
(4,3)						0.32776

$$P(s = \{4,3\}) = P(a_1) \times P(a_2) \times \cdots = ?$$

Transitions for a stochastic environment example

- If you are in (1,1) there is 80% chance that you will end up in (1,2) if you take action “Up”.
- But, there is 10% chance that you will stay in either (1,1) or move to (2,1) even if you took action Up
- By following this logic, we obtain the full transition matrix

		Intended Action				
		Up	Up	Right	Right	Right
(1,1)	1	0.1	0.02	0.026	.0284	.02462
(1,2)		0.8	0.24	0.258	0.2178	0.2462
(1,3)			0.64	0.88	0.346	0.2524
(2,1)		0.1	0.9	0.34	0.376	0.2824
(2,3)				0.512	0.1728	0.06224
…				…	…	…
(4,3)						0.32776

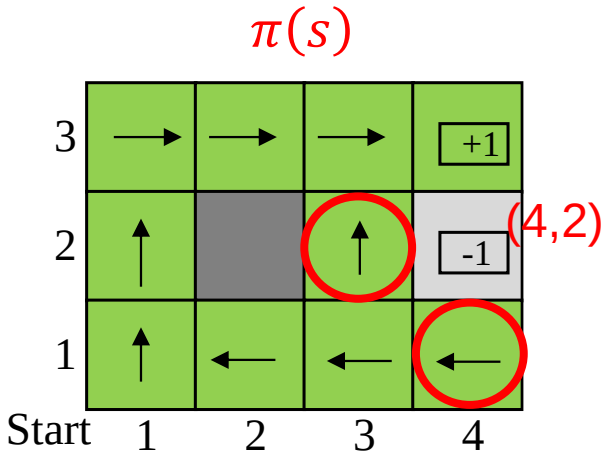
$$P(s = \{4,3\}) = P(a_1) \times P(a_2) \times \cdots = ?$$

Optimal (best) policy

But there can be many other policies as well!

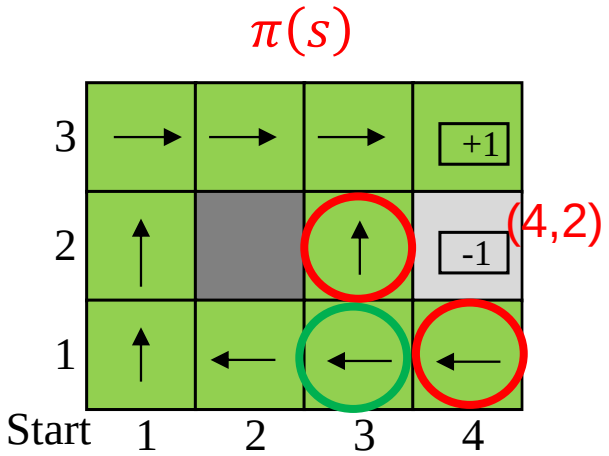
Optimal policy in our example

- What happens when the agent enters (3,2) or (4,1)?
  - Remember that actions are split 0.8 and 0.2 in stochastic environment
  - Increases the chance it will terminate at (4,2) which is not desirable



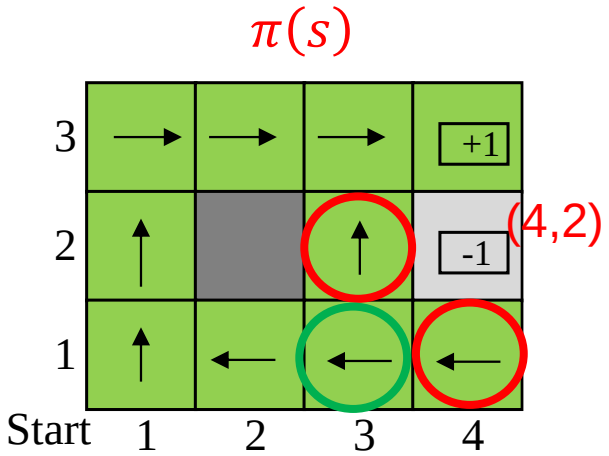
Optimal policy takes no risks

- Since the cost of taking an action (i.e., $R(s) = -0.04$) is small compared with the penalty "-1" for entering (4,2) by "*a chance*", the optimal policy for the state (3,1) is conservative
  - The policy recommends taking the long way round, rather than taking the shortcut and thereby risking entering (4,2)



Optimal policy takes no risks

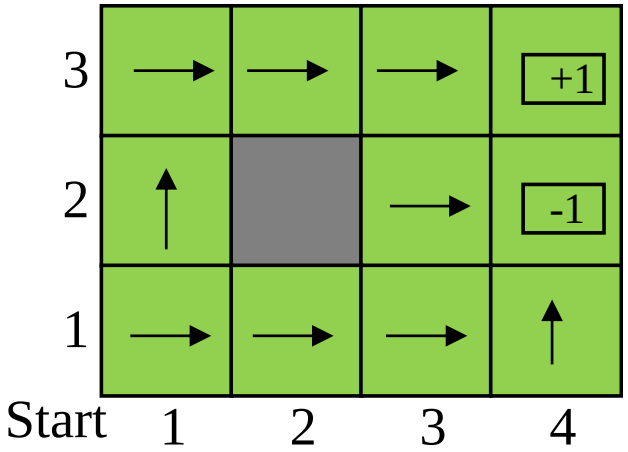
- Since the cost of taking an action (i.e., $R(s) = -0.04$) is small compared with the penalty "-1" for entering (4,2) by "*a chance*", the optimal policy for the state (3,1) is conservative
  - The policy recommends taking the long way round, rather than taking the shortcut and thereby risking entering (4,2)



Check what is the total utility if the agent reaches the +1 state after 10 steps!

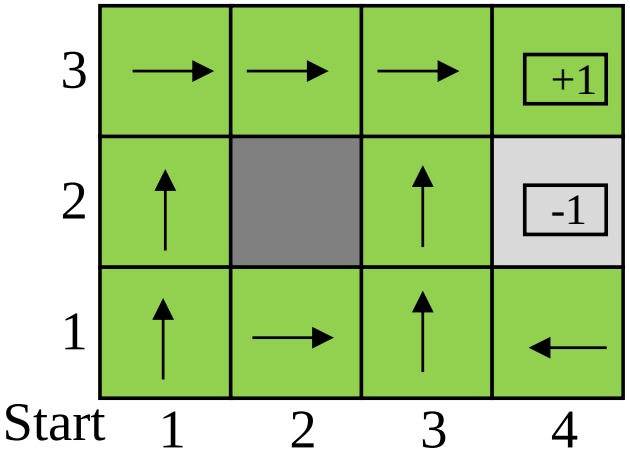
Balance of risk taken and reward

$$R(s) = -2$$



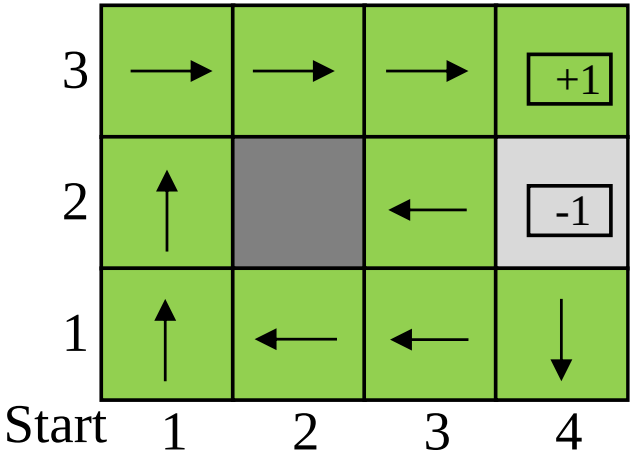
Balance of risk taken and reward

$$R(s) = -0.1$$

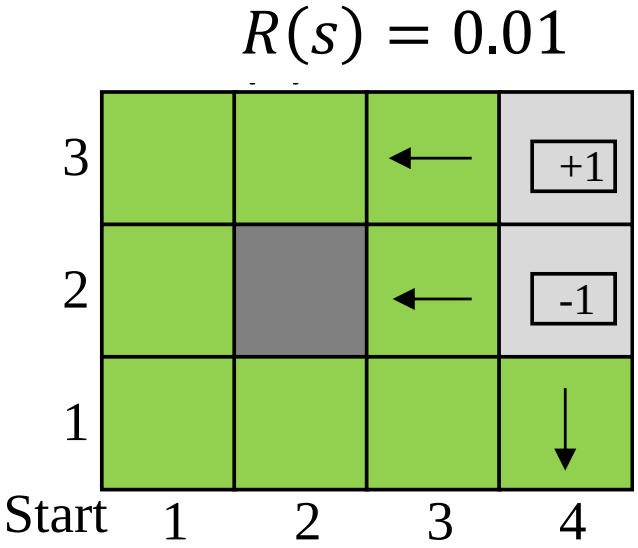


Balance of risk taken and reward

$$R(s) = -0.01$$



Balance of risk taken and reward



Remarks

- The careful balancing of risk and reward is a characteristic of MDPs that does not arise in deterministic search problems
- MDP is a characteristic of many real-world decision problems

- MDPs have been studied and utilized in several fields, including Communications, AI, Automation and Control theory, Operations research, Economics,…etc.

Utility functions over time

In the previous MDP example, the performance of the agent was measured by a sum of rewards for the states visited!
Can rewards be collected infinitely?

Remark – Sparse vs. auxiliary reward function

- Imagine the sparse reward function!

-5	-5	-5	Exit
-5	-5	-5	-5
-5	-5	-5	-5
Start	-5	-5	-5

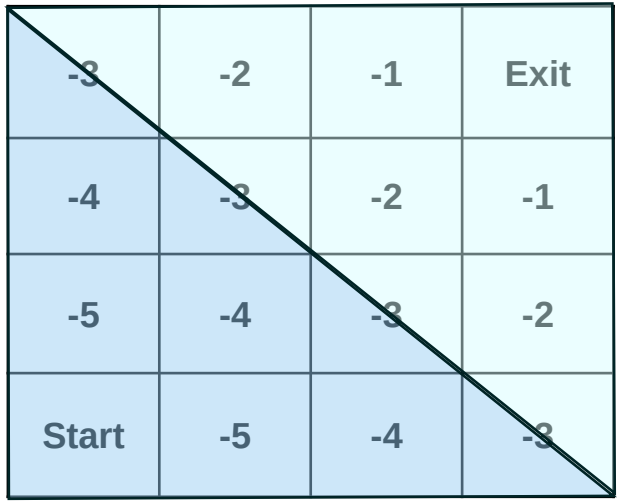
Remark – Sparse vs. auxiliary reward function

- Imagine the sparse reward function!
- **In principle, the agent learns that states closer to the Exit are better than those closer to its starting position**

-5	-5	-5	Exit
-5	-5	-5	-5
-5	-5	-5	-5
Start	-5	-5	-5

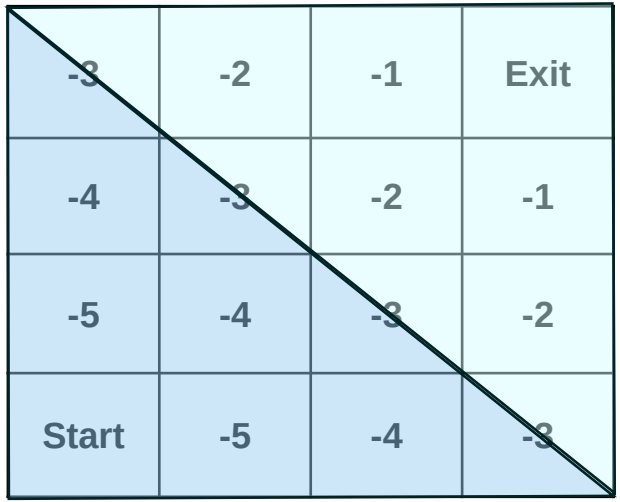
Remark – Sparse vs. auxiliary reward function

- Imagine the sparse reward function!
- The agent learns that states closer to the Exit are better than those closer to its starting position
- **So, it is important how much the reward function varies across states**
  - **With "sparse" reward function, i.e., sparse rewards, it is harder to learn**



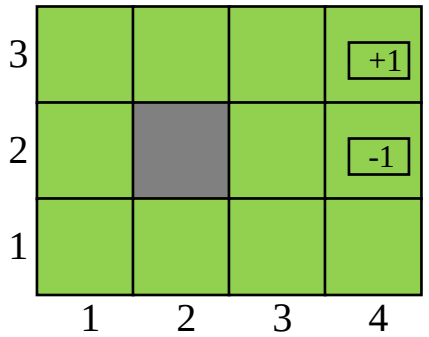
Remark – Sparse vs. auxiliary reward function

- Imagine the sparse reward function!
- The agent learns that states closer to the Exit are better than those closer to its starting position
- So, it is important is how much the reward function varies across states
  - With “sparse“ reward function, i.e., sparse rewards, it is harder to learn
  - **E.g., for sparse rewards across states, we define an “auxiliary” rewards that are added to the original rewards, so that the combined reward function is not flat – reward shaping**



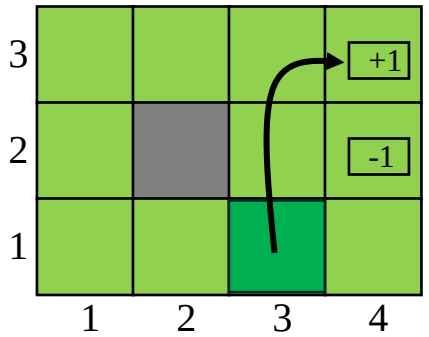
Finite vs. infinite horizon on an agent

- A finite horizon means that there is a *fixed* time *N* after which nothing matters



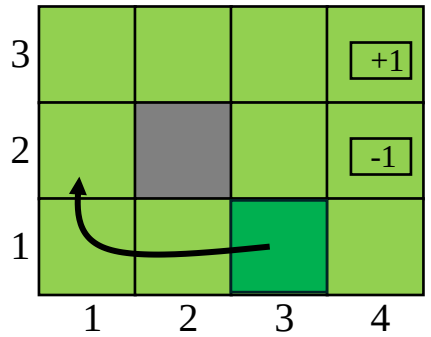
Finite vs. infinite horizon on an agent

- A finite horizon means that there is a *fixed* time *N* after which nothing matters
- For example, suppose an agent starts at (3,1)
  - For *N*=3, to have any chance of reaching the +1 state, the agent must head directly for it, and the optimal action is to go *Up*.



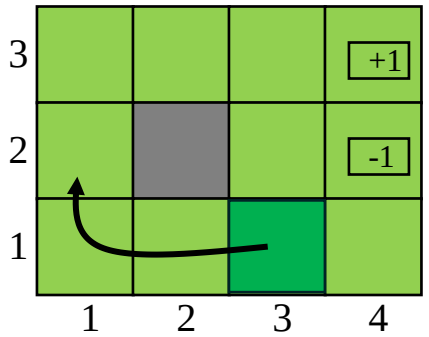
Finite vs. infinite horizon on an agent

- A finite horizon means that there is a *fixed* time *N* after which nothing matters
- For example, suppose an agent starts at (3,1)
  - For *N* = 3, to have any chance of reaching the +1 state, the agent must head directly for it, and the optimal action is to go *Up*.
  - For *N* = 100, there is plenty of time to take the safe route by going *Left*.



Finite vs. infinite horizon on an agent

- A finite horizon means that there is a *fixed* time *N* after which nothing matters
- For example, suppose an agent starts at (3,1)
  - For *N* = 3, to have any chance of reaching the +1 state, the agent must head directly for it, and the optimal action is to go *Up*.
  - For *N* = 100, there is plenty of time to take the safe route by going *Left*.



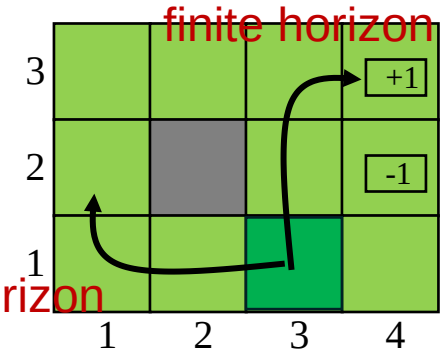
Remarks

- With a finite horizon, the optimal action in a given state could change over time
  - The optimal policy for a finite horizon is nonstationary!

- With infinite horizon, on the other hand, there is no reason to behave differently in the same state at different times
  - The optimal action depends only on the current state, and the optimal policy is stationary!

- Policies for the infinite-horizon case are simpler than those for the finite-horizon case

finite horizon



infinite horizon

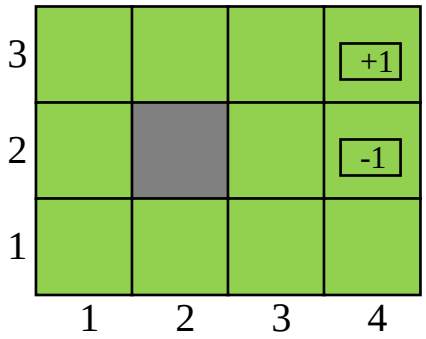
Definition of utility function over time

- The utility function for a sequences of states (i.e., environment histories) is defined as $U(\mathbf{s})$
  - $\mathbf{s} = [s_0, s_1, \dots, s_t, \dots]$ is a state vector (i.e., finite or infinite)
  - Utility function $U^{\pi}(s_t)$ defines expected return when starting in s_t and following policy π
- How to assign utilities to sequences?

Utility assignment to sequences of states with Additive Rewards

- The utility of a state sequence with **additive rewards** is

$$U(\mathbf{s}) = R(s_0) + R(s_1) + \cdots + R(s_t) + \cdots$$

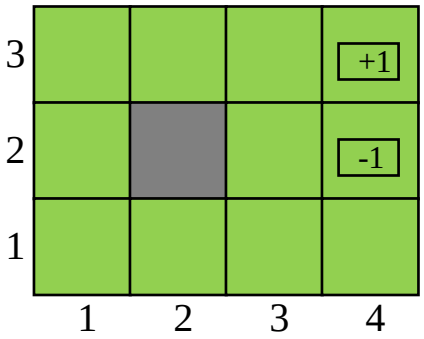


Utility assignment to sequences of states with Additive Rewards

- The utility of a state sequence with **additive rewards** is

$$U(\mathbf{s}) = R(s_0) + R(s_1) + \cdots + R(s_t) + \cdots$$

- **Note 1**: If the environment does not contain a terminal state, or if the agent never reaches one, then all environment histories will be infinitely long
  - Utilities $U(\mathbf{s})$ with additive rewards will generally be infinite.

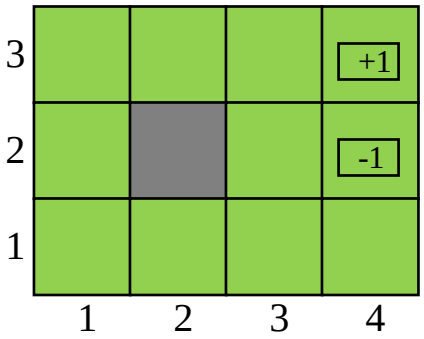


Utility assignment to sequences of states with Additive Rewards

- The utility of a state sequence with **additive rewards** is

$$U(\mathbf{s}) = R(s_0) + R(s_1) + \cdots + R(s_t) + \cdots$$

- **Note 1**: If the environment does not contain a terminal state, or if the agent never reaches one, then all environment histories will be infinitely long
  - Utilities $U(\mathbf{s})$ with additive rewards will generally be infinite.
- **Note 2**: If all environment histories are infinite (no terminal state reached), using additive rewards results in comparing ∞ sequences → Computational problem!!!

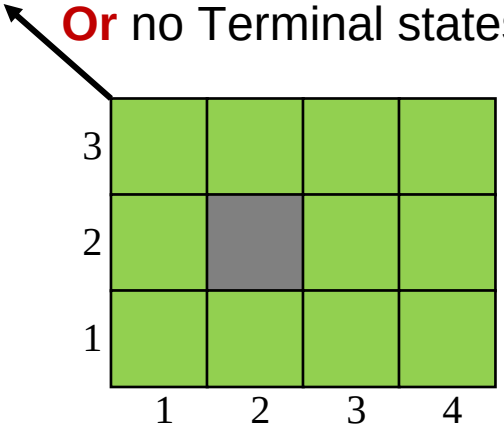


Utility assignment to sequences of states with discounted rewards

- **Discounted rewards**: The utility of a state sequence is

$$U(s) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots$$

Increase the world size from 4×3 to $4 \times \infty$
Or no Terminal states



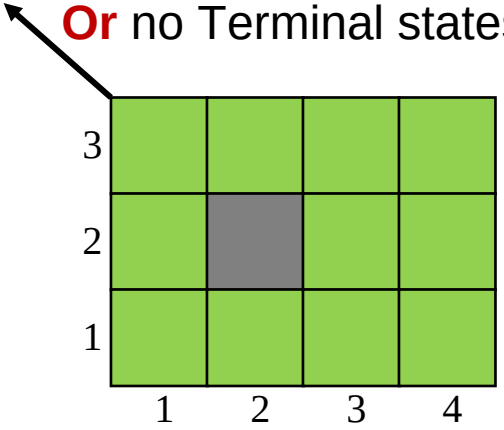
Utility assignment to sequences of states with discounted rewards

- **Discounted rewards**: The utility of a state sequence is

$$U(s) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots$$

- The **discount factor** γ is a number between 0 and 1, describing the preference of an agent for current rewards over future rewards
- E.g.,
  - When γ is close to 0, rewards in the future are viewed as not relevant
  - When γ is close to 1, discounted rewards are becoming equivalent to additive rewards

Increase the world size from 4×3 to $4 \times \infty$
Or no Terminal states



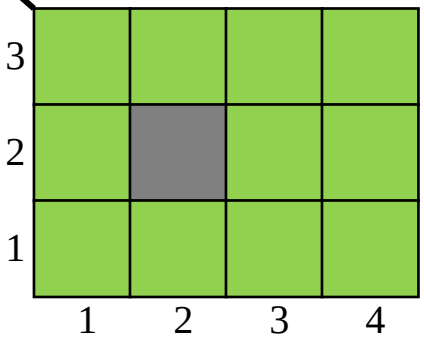
Compact form of utility with discounted rewards

- The utility of a state sequence with **discounted rewards** is given by

$$U(s) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \cdots$$

$$= \sum_{t=0}^{\infty} \gamma^t R(s_t)$$

Increase the world size from 4×3 to $4 \times \infty$
Or no Terminal states



Utility with discounted rewards of an infinite sequence is finite

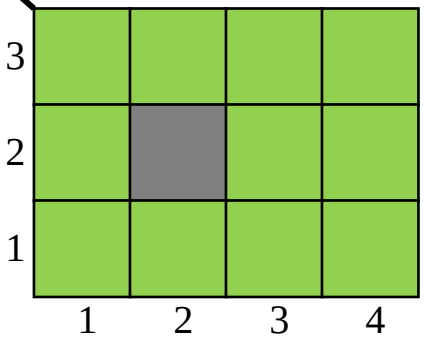
- We use the standard formula for an infinite geometric series!
- When $\gamma < 1$, and rewards are counded by $\pm R_{max}$ we obtain

$$U(\mathbf{s}) = \sum_{t=0}^{\infty} \gamma^t R(s_t) \leq \sum_{t=0}^{\infty} \gamma^t R_{max}$$

$$= \frac{R_{max}}{1-\gamma}$$

- The utility is finite!

Increase the world size from 4×3 to $4 \times \infty$
Or no Terminal states



Practical situation with limited sequence

- **If the environment contains terminal states s_t** or the agent is guaranteed to **reach one**, the finite utility is sufficient:

$$U(\mathbf{s}) = R(s_0) + R(s_1) + \cdots R(s_t)$$

Practical situation with limited sequence

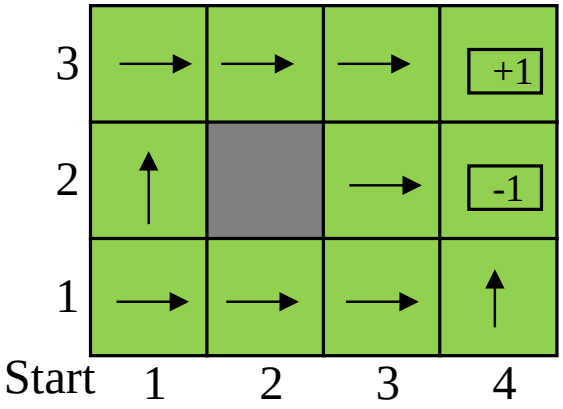
- **If the environment contains terminal states s_t** or the agent is guaranteed to **reach one**, the finite utility is sufficient:

$$U(\mathbf{s}) = R(s_0) + R(s_1) + \cdots R(s_t)$$

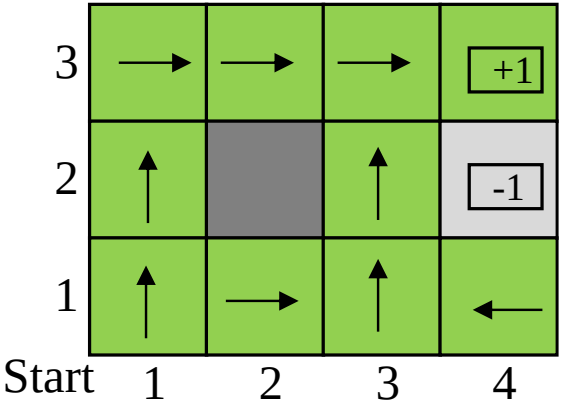
- A policy that is guaranteed to reach a terminal state is called a proper policy
  - With proper policies, we use additive rewards (i.e., $\gamma = 1$).
  - No need to compare infinite sequences! → SUPER!

Examples of proper policies

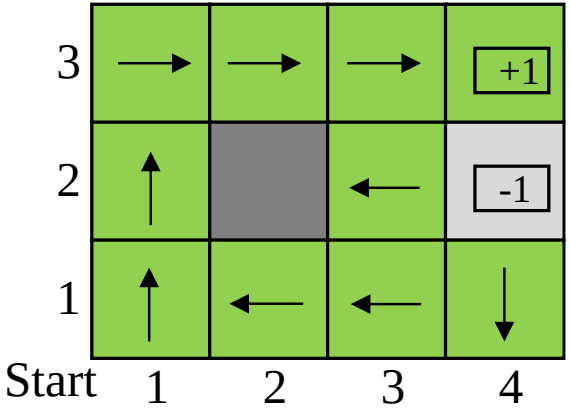
$R(s) = -2$



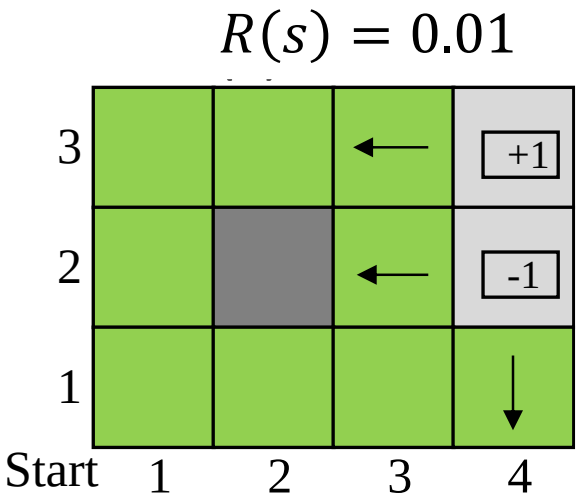
$R(s) = -0.1$



$R(s) = -0.01$



Example of improper policy

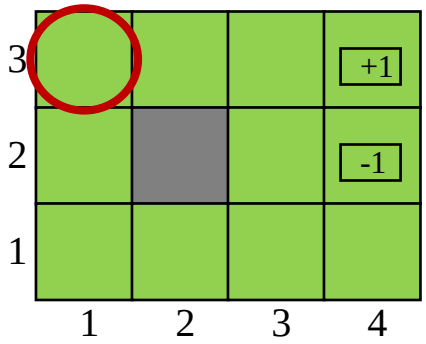


- This policy gains infinite total reward by staying away from the terminal states when the reward for the nonterminal states is positive.

Average reward per time step for infinite sequences

- Assume that state **(1,3)** has a reward of 0.1 while the other nonterminal states have a reward of 0.01
  - Consequently, a policy that targets staying in state **(1,3)** will have higher average reward than one that stays elsewhere.

- Note: average reward algorithms are out of our scope!



Optimal policies and utilities of states

Q: How to compare different policies, i.e., output of an agent?

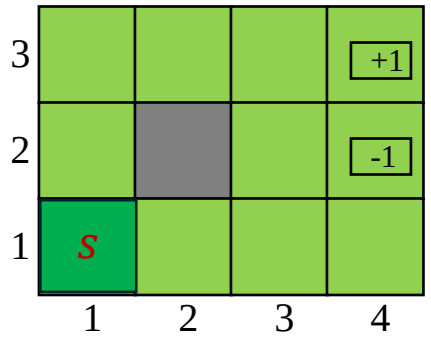
A: We compare policies by comparing the *expected* utilities obtained when executing different policies

Expected utility function

- The expected utility obtained by executing π starting from state $s = s_0$ is given by

$$U^{\pi}(s = s_0) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]_{w.r.t.\ s,\pi}$$

where the expectation $E[\cdot]$ is with respect to the probability distribution over state sequences determined by s and π



Expected utility function

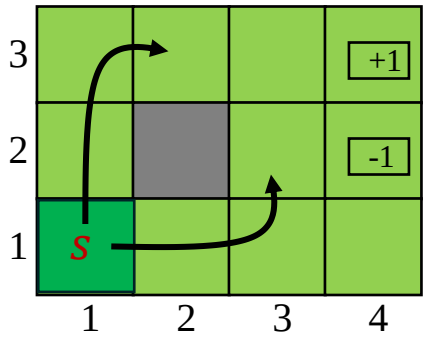
- The expected utility obtained by executing π starting from state $s = s_0$ is given by

$$U^{\pi}(s = s_0) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t)\right]_{w.r.t.\ s,\pi}$$

where the expectation $E[\cdot]$ is with respect to the probability distribution over state sequences determined by s and π

- Now, out of all the policies the agent could choose to execute starting in s, one (or more) will have higher expected utilities than all the others (optimal policy):

$$\pi^*(s) = \arg\max_{\pi} U^{\pi}(s = s_0)$$



True utility of a state

- If the agent executes an optimal policy $\pi^*(s)$ in state s, the true utility of a state is defined as the expected sum of discounted rewards:

$$U^{\pi^*}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s)\right]$$

- Henceforth, we simplify notation as

$$U^{\pi^*}(s) = U(s)$$

What is the relationship between a Policy, Reward and Utility?

Reward vs. Utility

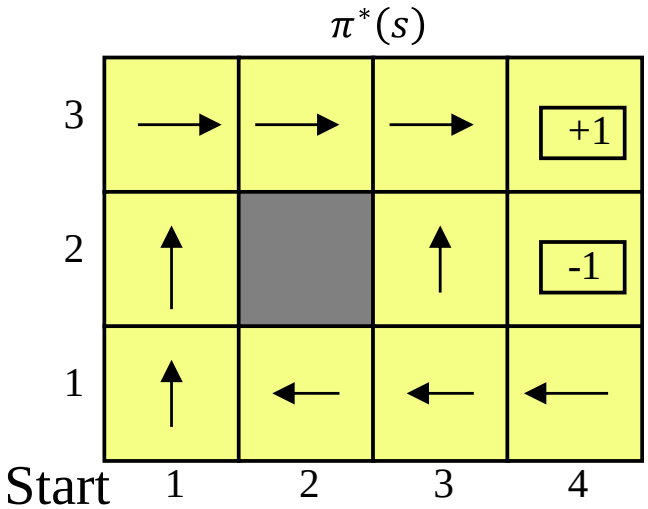
- $R(s)$ is the "*short term*" reward for being in the state s

Reward vs. Utility

- $R(s)$ is the "*short term*" reward for being in the state s
- $U^{\pi}(s)$ is the "*long term*" total reward starting in the state s
  - The utilities are higher for states closer to the Exit, because fewer steps are required to reach the Exit (+1)

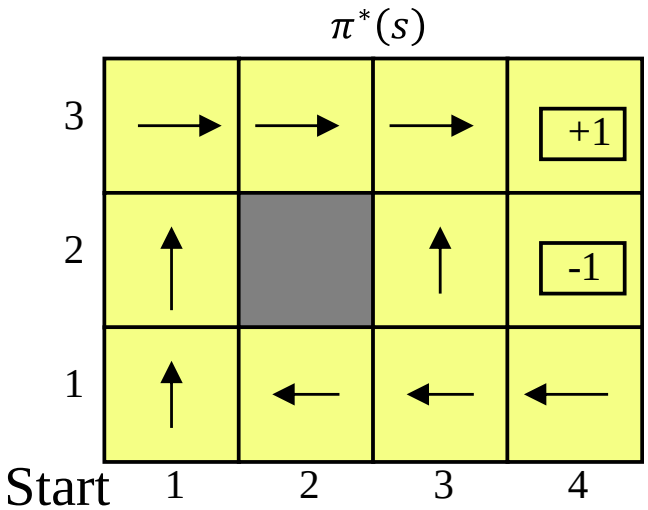
Policy vs. Utility

- We assume optimal policy for a 4x3 example, with $\gamma = 1$ and $R(s) = -0.04$

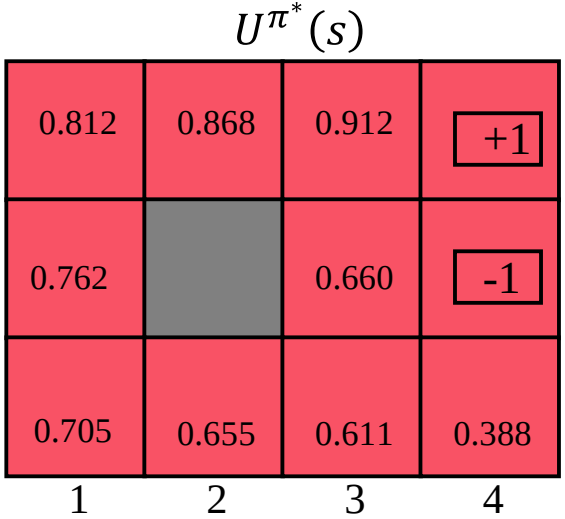


Policy vs. Utility

- We assume optimal policy for a 4x3 example, with $\gamma = 1$ and $R(s) = -0.04$



- For transitions to nonterminal states the utility is given by



Remarks

- If the agent has a full action-state policy $\pi(s)$, then no matter what the outcome of any action, the agent will always know what to do next.
- A policy represents the agent function explicitly and is therefore a description of a simple reflex agent, computed from the information used for a utility-based agent.
- Given π^* for all states, the agent decides what to do by observing current percept, which tells it the current state s, and then executing the action $\pi^*(s)$
- Remember that $\pi^*(s)$ is a policy, so it recommends an action for every state;
  - its connection with s in particular is that it's an optimal policy when s is the starting state.

How to select among Policies?

Utility as a metric for action selection given a state

- We compare Utilities!

Utility as a metric for action selection given a state

- We compare Utilities!
  - **Rule: select the action that maximizes the expected utility given by**

$$\pi^*(s) = \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s,a)U(s') \qquad (17.4)$$

Utility as a metric for action selection given a state

- We compare Utilities!
  - **Rule:** select the action that maximizes the expected utility given by

$$\pi^*(s) = \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s,a)U(s') \qquad (17.4)$$

- **What this equation does?**

Utility as a metric for action selection given a state

- We compare Utilities!
  - **Rule:** select the action that maximizes the expected utility given by

$$\pi^*(s) = \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s,a)U(s') \qquad (17.4)$$

- **What this equation does?**
  - **We simply allocate the weighting factors to each transition $(s'|s,a)$ within the policy.**

Remark

- Given the optimal policy criteria:

$$\pi^*(s) = \underset{\boldsymbol{a} \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s, \boldsymbol{a}) \boldsymbol{U}(s')$$

- What can we change?

Remark

- Given the optimal policy criteria:

$$\pi^*(s) = \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s,a)\boldsymbol{U}(s')$$

- What can we change?

- Intuition leads us to two approaches:
  - **Update of of utility/value function**

Remark

- Given the optimal policy criteria:

$$\pi^*(s) = \underset{\boldsymbol{a} \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s, \boldsymbol{a}) U(s')$$

- What can we change?

- Intuition leads us to two approaches:
  - Update of utility/value function
  - **Update of actions/policy function**

Remark

- Given the optimal policy criteria:

$$\pi^*(s) = \underset{\boldsymbol{a} \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s, \boldsymbol{a}) U(s')$$

- What can we change?

- Intuition leads us to two approaches:
  - Update of utility/value function
  - Update of actions/policy function

Now we have to design algorithms for finding optimal policies!!!

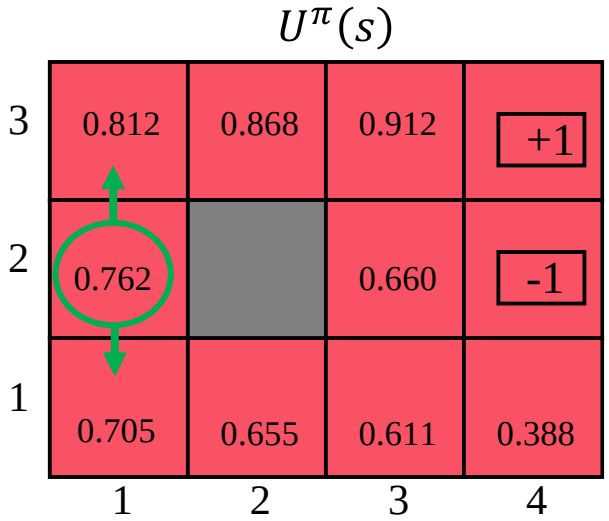
Value iteration algorithm

Recap

- The utility of being in a state $s = s_0$ is the expected sum of discounted rewards from that state given by

$$U(s = s_0) = \sum_{t=0}^{\infty} \gamma^t R(s_t)$$

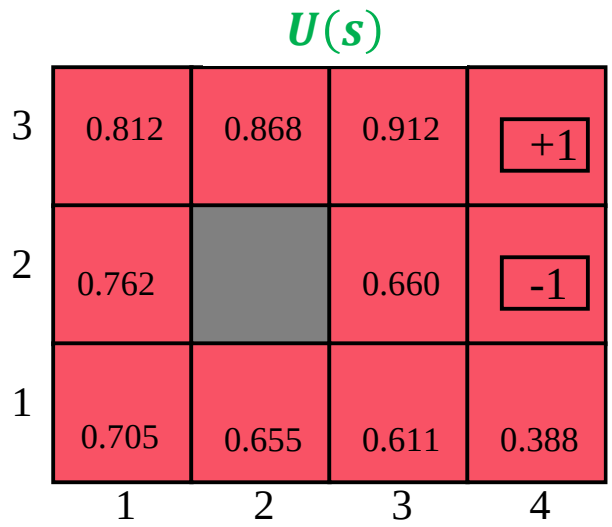
- Hence, there is a direct relationship between the utility of a state s and the utility of its neighboring states ☺
  - **Keep this in mind!!!**

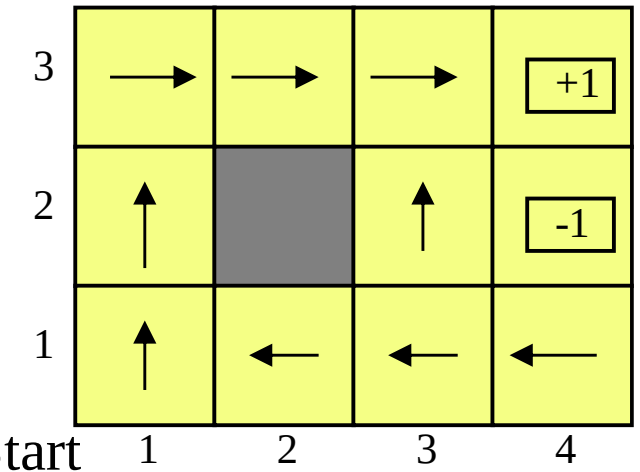


Basic principle

- We calculate the utility of each state and then use the state utilities to select an optimal action in each state.

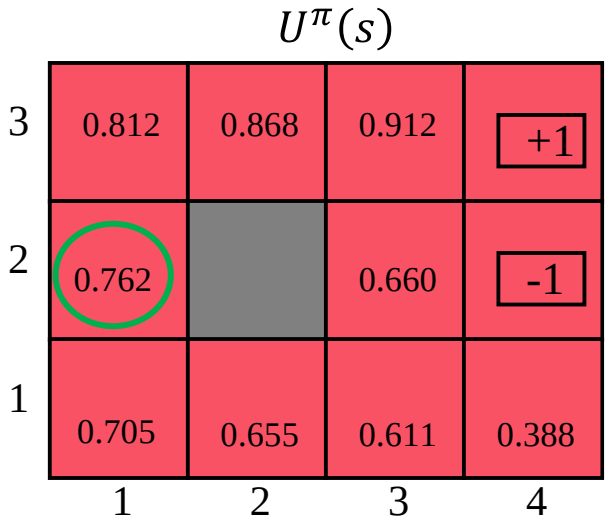
$$\pi^*(s) = \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s,a) U(s')$$





Direct Utility Estimation with Bellman equation

- Recall a direct relationship between the utility of a state and the utility of its neighbors!

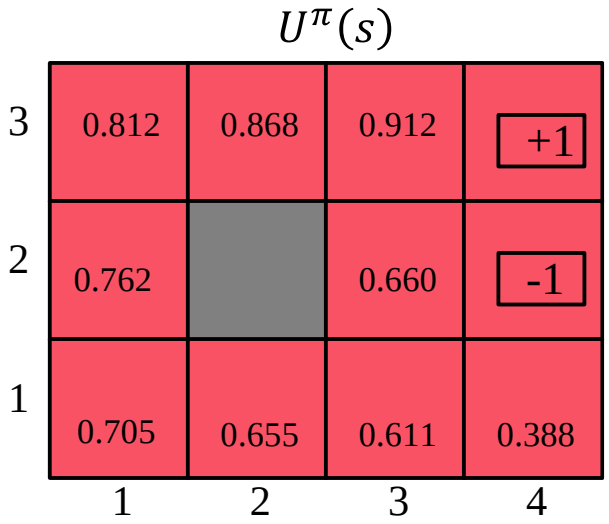


Direct Utility Estimation with Bellman equation

- Recall a direct relationship between the utility of a state and the utility of its neighbors!
- The utility of each state equals its current reward plus the expected utility of its successor states

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|s,a)U(s')$$

Bellman equation

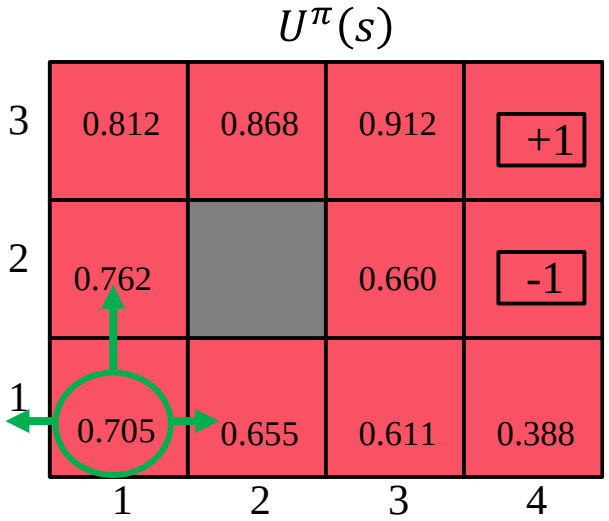


Direct Utility Estimation with Bellman equation

- Recall a direct relationship between the utility of a state and the utility of its neighbors!
- The utility of each state equals its current reward plus the expected utility of its successor states

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|s,a)U(s')$$

Bellman equation



Let's compute the utility for $s = (1,1)$

Let’s see our example

- For state (1,1) we have:

$$U(1,1) = R(1,1) + \gamma \max_{a \in A(1,1)} \sum_{s'} P(s'|(1,1), a) U(s')$$

$$= -0.04 + \gamma \max \begin{bmatrix} 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1) \end{bmatrix} \begin{matrix} (Up) \\ (Left) \\ (Down) \\ (Right) \end{matrix}$$

Let's see our example

- For state (1,1) we have:

$$U(1,1) = R(1,1) + \gamma \max_{a \in A(1,1)} \sum_{s'} P(s'|(1,1), a) U(s')$$

$$= -0.04 + \gamma \max \begin{bmatrix} 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1) & (Up) \\ 0.9U(1,1) + 0.1U(1,2) & (Left) \\ & (Down) \\ & (Right) \end{bmatrix}$$

Let's see our example

- For state (1,1) we have:

$$U(1,1) = R(1,1) + \gamma \max_{a \in A(1,1)} \sum_{s'} P(s'|(1,1), a) U(s')$$

$$= -0.04 + \gamma \max \begin{bmatrix} 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1) & (Up) \\ 0.9U(1,1) + 0.1U(1,2) & (Left) \\ 0.9U(1,1) + 0.1U(2,1) & (Down) \\ & (Right) \end{bmatrix}$$

Let's see our example

- For state (1,1) we have:

$$U(1,1) = R(1,1) + \gamma \max_{a \in A(1,1)} \sum_{s'} P(s'|(1,1), a) U(s')$$

$$= -0.04 + \gamma \max \begin{bmatrix} 0.8U(1,2) + 0.1U(2,1) + 0.1U(1,1) & (Up) \\ 0.9U(1,1) + 0.1U(1,2) & (Left) \\ 0.9U(1,1) + 0.1U(2,1) & (Down) \\ 0.8U(2,1) + 0.1U(1,2) + 0.1U(1,1) & (Right) \end{bmatrix}$$

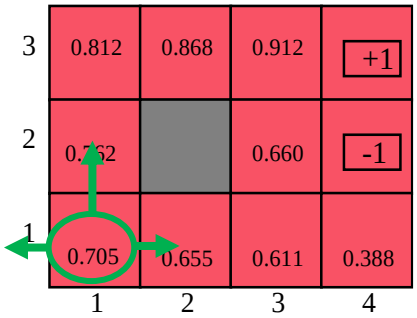
- Now we use the values from the utility table to compute the state utility function!

Best action selection

- For state (1,1) we have:

$$U(1,1) = -0.04 + \gamma \max \begin{bmatrix} 0.8 \times 0.762 + 0.1 \times 0.655 + 0.1 \times 0.705 \\ 0.9 \times 0.705 + 0.1 \times 0.762 \\ 0.9 \times 0.705 + 0.1 \times 0.655 \\ 0.8 \times 0.655 + 0.1 \times 0.762 + 0.1 \times 0.705 \end{bmatrix} \begin{matrix} (Up) \\ (Left) \\ (Down) \\ (Right) \end{matrix}$$

$$= -0.04 + \gamma \max \begin{bmatrix} 0{,}7456 \\ 0{,}0483489 \\ 0{,}04155975 \\ 0{,}6707 \end{bmatrix} \begin{matrix} (Up) \\ (Left) \\ (Down) \\ (Right) \end{matrix}$$



- Out of 4 actions we calcuate that "*Up*" is the highest!

Best action selection

- For state (1,1) we have:

$$U(1,1) = -0.04 + \gamma \max \begin{bmatrix} 0{,}7456 \\ 0{,}0483489 \\ 0{,}04155975 \\ 0{,}6707 \end{bmatrix}$$
$$= -0.04 + \gamma \times 0{,}7456$$

- Now, we can continue with computation for all other states!

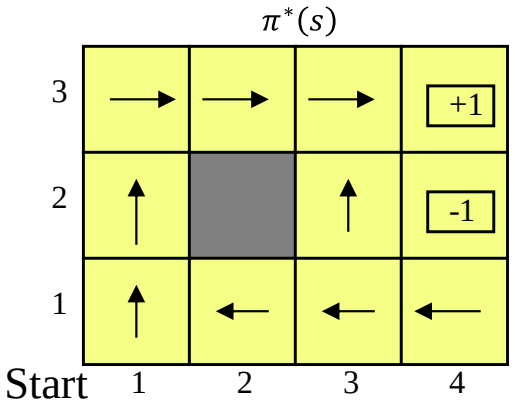
Solution to Bellman equations

- To solve the Bellman equations for s_t possible states there exist t unique Bellman equations!

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|s,a)U(s')$$

- Q: How many states do we have in our 3x4 problem?

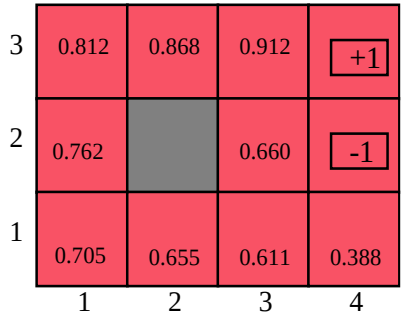
- But the problem is that these equations are nonlinear:
  - Why?
  - How to solve them?



Iterative solution for VALUE-ITERATION algorithm

- We start with arbitrary initial values for the utilities;
- Update the utility of each state from the utilities of its neighbors
  - Compute the right-hand side of the Bellman equation, and plug it into the left-hand side
  - For the utility value for state s at the ith iteration, $U_i(s)$, a Bellman update iteration step is given by:

$$U_{i+1}(s) \leftarrow R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|s,a) U_i(s')$$



- We repeat this until we reach a steady state (changes are less than a set performance measure)
  - apply updates simultaneously to all the states at each iteration

Value-Iteration Algorithm

function VALUE-ITERATION(mdp, ε) **returns** $U(s)$

inputs: an $mdp[\mathbf{S}, \boldsymbol{A}(s), P(s'|s,a)]$, $R(s)$, γ and performance error ε

variables $U(s) = 0,\ U_{i+1} = 0$, the maximum utility change δ

repeat $\forall i$

$U \leftarrow U'; \delta \leftarrow 0$

for each s **in** S **do**

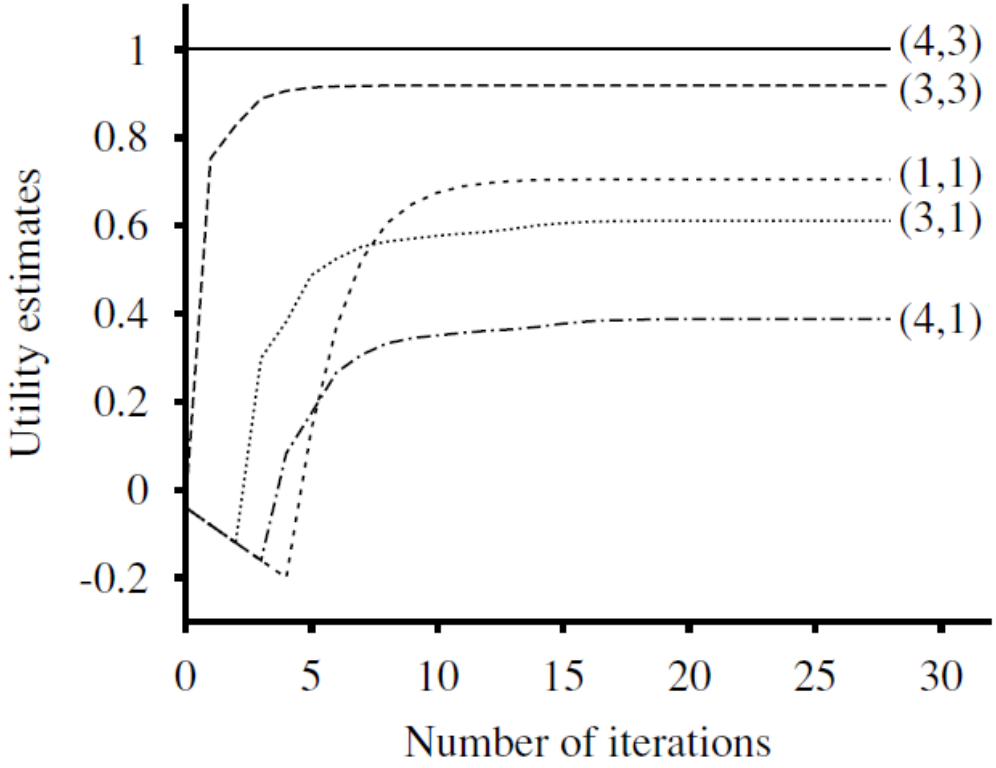
$$U'(s) \leftarrow R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|s,a) U(s')$$

if $|U'(s) - U(s)| > \delta$ **then** update $\delta \leftarrow |U'(s) - U(s)|$

until $\delta \frac{\gamma}{1-\gamma} < \varepsilon$

return $U(s)$

Utility estimation for the 4 × 3 world



3	0.812	0.868	0.912	+1
2	0.762		0.660	-1
1	0.705	0.655	0.611	0.388
	1	2	3	4

Policy Iteration algorithm

Principle of the Policy Iteration algorithm

- **Start** with an initial policy $\pi(s)$, while the following two steps define the algorithm:

Principle of the Policy Iteration algorithm

- **Start** with an initial policy $\pi(s)$, and follow next two steps:
  - ***Policy evaluation***:
    - Given a policy π_i, compute the utility $U_i(s) = U^{\pi_i}$ for each state when $\pi_i(s)$ is executed

Principle of the Policy Iteration algorithm

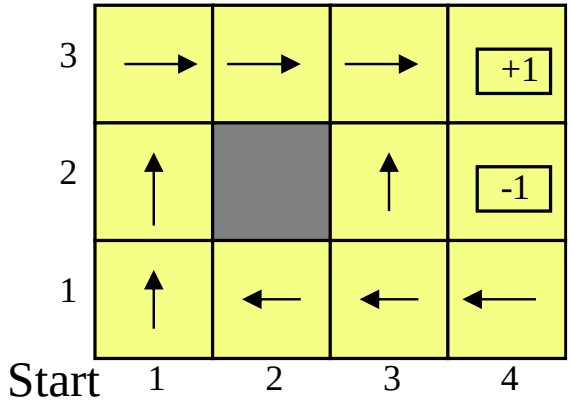
- **Start** with an initial policy $\pi(s)$, and follow next two steps:
  - ***Policy evaluation***:
    - Given a policy π_i, compute the utility $U_i(s) = U^{\pi_i}$ for each state when $\pi_i(s)$ is executed
  - ***Policy improvement***:
    - Compute a new *maximum expected utility* policy $\pi_{i+1}(s)$ by using one-step

$$\pi_{i+1}(s) \leftarrow \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s,a) U_i(s')$$

- **End** *When the policy improvement step yields no change in the utilities*

An example of 3×4 world

- Let's assume an optimal policy $\pi^*(s)$ of our 3×4 example, where at the ith iteration, the policy π_i specifies the action $\pi_i(s)$ in state s

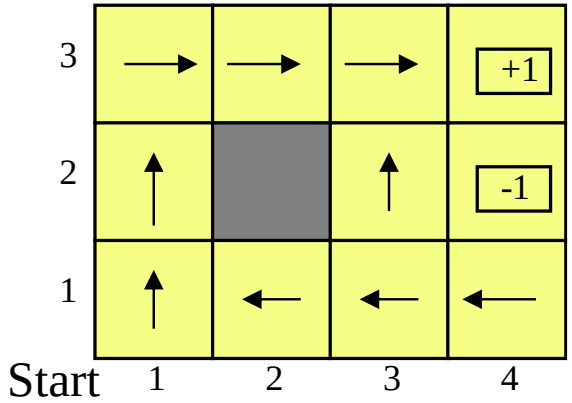


An example of 3×4 world

- Let's assume an optimal policy $\pi^*(s)$ of our 3×4 example, where at the ith iteration, the policy π_i specifies the action $\pi_i(s)$ in state s

- This simplifies the computation of the Bellman equation relating the utility of s (under policy $\pi_i(s)$) to the utilities of its neighbors $U_i(s')$ as follows:

$$U_i(s) = R(s) + \gamma \sum_{s'} P(s'|s, \pi_i(s)) U_i(s')$$



Example of optimal policy for 3×4 world

- For the optimal policy, we have:

$$\begin{cases} \pi_i(1,1) = Up \\ \pi_i(1,2) = Up \\ \pi_i(1,3) = Right \\ \quad\quad \vdots \end{cases}$$

Example of optimal policy for 3×4 world

- For the optimal policy, we have:

$$\begin{cases} \pi_i(1,1) = Up \\ \pi_i(1,2) = Up \\ \pi_i(1,3) = Right \\ \quad\quad \vdots \end{cases}$$

- Thus, the simplified Bellman equations are:

$$U_i(1,1) = -0.04 + 0.8U_i(1,2) + 0.1U_i(1,1) + 0.1U_i(2,1)$$
$$U_i(1,2) = -0.04 + 0.8U_i(1,3) + 0.2U_i(1,2)$$
$$\vdots$$

Remark

- In this case, the simplified Bellman equations are linear (no “max” operator)

$$U_i(1,1) = -0.04 + 0.8U_i(1,2) + 0.1U_i(1,1) + 0.1U_i(2,1)$$

$$U_i(1,2) = -0.04 + 0.8U_i(1,3) + 0.2U_i(1,2)$$

$$\vdots$$

- So, for n states, we have n linear equations with n unknowns
  - Solution in time $O(n^3)$ by standard linear algebra
  - For small state spaces, policy evaluation using exact solution methods is often the most efficient approach.

Note: For large state spaces, $O(n^3)$ time might be prohibitive, but practically it is not necessary to do exact policy evaluation

Modified policy iteration algorithm

- Since it is not necessary to do an exact policy evaluation, we perform several value iteration steps with a fixed policy to compute a reasonably good approximation of the utilities
- Thus, we compute simplified Bellman update as:

$$U_{i+1}(s) \leftarrow R(s) + \gamma \sum_{s'} P(s'|s, \pi_i(s)) U_i(s')$$

repeated multiple times to compute the next utility estimate!

- This is Modified Policy Iteration algorithm!

Policy-Iteration Algorithm

function POLICY-ITERATION(mdp) **returns** π
  inputs: an $mdp[S, A(s), P(s'|s,a)]$
  variables $U(s) = 0$, random initial policy π
  repeat
    $U(s) \leftarrow EVALUATE\_POLICY(\pi, U, mdp)$
    $unchanged? \leftarrow$ true
      for each s **in** S **do**

$$\textbf{if } \max_{a \in A(s)} \sum_{s'} P(s'|s,a)U(s') > \sum_{s'} P(s'|s,\pi[s])U[s'] \textbf{ then do}$$

$$\pi(s) \leftarrow \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s'|s,a)U[s']$$

    $unchanged? \leftarrow$ false
  until $unchanged?$
  return π

Final remark on partial observability

- If the system state cannot always be determined, our policy/action outcomes are not fully observable!
- We can modify our approach as a Partially Observable MDP!
  - But this is out of the scope of this course!
  - Those who are interested refer to Section 17.4

Thank you!

DSP Chair for Distributed Signal Processing | RWTH AACHEN UNIVERSITY

DSP Chair for Distributed Signal Processing | RWTH AACHEN UNIVERSITY